

A Project Report

Submitted To

The Department of Computer Science and Engineering, Metropolitan University, in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

On

Sarabo: An Electronics Repair Service Management Platform

Submitted By

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)

Under The Supervision of

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

Submission Date: 26 October 2026



STUDENT'S DECLARATION

I, the undersigned, solemnly declare that the project report entitled **"Sarabo: An Electronics Repair Service Management Platform"** is based on my original project work carried out during the course of study under the supervision of **Shrabanti Chowdhury**. The analysis, design, implementation, testing, and documentation presented in this report were completed as an individual final year project, except where the work of others has been properly cited.

I further declare that, to the best of my knowledge and belief, this report has not been submitted elsewhere in substantially the same form for the requirement of any other degree. All external literature, standards, documentation, libraries, and technologies used in the project have been acknowledged in the reference section.

Jahid Hasan

ID No: 222-115-116

Batch: 57 (C)



SUPERVISOR'S RECOMMENDATION

The project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), is a record of project work carried out under my supervision. I hereby recommend that this project report is satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering and may be processed for further evaluation.

Shrabanti Chowdhury

Senior Lecturer

Department of Computer Science and Engineering

Metropolitan University, Sylhet

(Supervisor)



LETTER OF APPROVAL

This is to certify that the project work entitled **"Sarabo: An Electronics Repair Service Management Platform"** submitted by **Jahid Hasan**, ID No: 222-115-116, Batch: 57 (C), has been studied and approved by the project examination committee as satisfactory in partial fulfillment of the requirements for the degree of Bachelor of Science in Computer Science and Engineering.

Shrabanti Chowdhury

Senior Lecturer, Department of Computer Science and Engineering

Supervisor

Nabila Zannat Rifa

Senior Lecturer

Reviewer 1

Md. Fahmidur Rahman Sakib

Senior Lecturer

Reviewer 2

Abstract

Electronics repair in Bangladesh is still largely informal: customers hand over a phone, laptop or appliance without knowing the final price, cannot see where it is, and have little recourse if something goes wrong, while technicians have no structured way to find nearby work or get paid reliably. This project presents **Sarabo**, a web-based electronics repair service management platform that turns this process into a tracked, priced and auditable service.

Sarabo connects three roles. A *Customer* describes the problem (optionally letting AI fill the form from Bangla, English or Banglish text and a damage photo), books a pickup window, compares offers from qualified technicians in the same region, pays an inspection fee that counts toward the final price, approves or rejects the itemised quote, pays, tracks progress and reviews the technician. A *Technician* finds work on an open jobs board, sends offers guided by a data-driven price guide, inspects, quotes, repairs and withdraws earnings through bKash. An *Admin* approves technicians, maintains a versioned catalogue of 45 services, handles missed pickups and refunds, moderates reviews and reads reports with an AI-written summary.

The client uses React 19 and Vite; the server uses Node.js, Express 5 and MongoDB, with Firebase Authentication, Stripe payments and refunds, private Supabase storage and Google Gemini. Every money- or assignment-changing step runs in a database transaction, Stripe's signed webhook is the single source of payment truth, and a 90/10 technician/platform settlement is frozen at payment time. The system is deployed on Vercel and Firebase Hosting and was verified with 2,595 passing automated server checks, 58 passing browser tests, lint and build checks, and continuous integration.

Keywords: Electronics repair, service marketplace, workflow management, React, Express, MongoDB, Stripe, role-based access control, generative AI.

Acknowledgements

I express my sincere gratitude to my supervisor, **Shrabanti Chowdhury**, Senior Lecturer, Department of Computer Science and Engineering, Metropolitan University, for her guidance, feedback, and encouragement throughout this final year project. Her supervision helped me refine Sarabo from an initial application idea into a structured service management platform.

I am also grateful to the faculty members of the Department of Computer Science and Engineering for the academic foundation that supported this work, and to the reviewers, **Nabila Zannat Rifa** and **Md. Fahmidur Rahman Sakib**, for evaluating the project and for their feedback.

Finally, I acknowledge the open-source communities and documentation maintainers behind React, Express, MongoDB, Firebase, Stripe, Supabase, Google Gemini, TanStack Query, Tailwind CSS, Playwright and the other tools used in Sarabo. Their work made it possible to build and test a full-stack platform of this scope as an individual project.

Contents

Abstract I
Acknowledgements II
Table of Contents III
List of Tables VIII
List of Figures X
Listings XI

1 Introduction **1**

1.1 Project Overview 1
1.2 Problem Statement 2
1.3 Motivation 2
1.4 Objectives 3
1.5 Scope 3
1.6 Individual Contribution 4
1.7 Report Organization 4

2 Background and Related Work **5**

2.1 Web-Based Service Management 5
2.2 Single-Page Application Architecture 5
2.3 RESTful API and Express 5
2.4 Document Database and Transactions 6
2.5 Authentication and Authorization 6
2.6 Online Payment Processing 6
2.7 Private Object Storage 7

2.8 Large Language Models in Applications 7
2.9 Usability and Accessibility 7
2.10 Related Systems 7

3 Requirements Analysis **9**
3.1 Stakeholders and Roles 9
3.2 Functional Requirements 9
3.3 Non-Functional Requirements 12
3.4 Service Catalogue 12
3.5 Use-Case View 13

4 System Design and Architecture **14**
4.1 High-Level Architecture 14
4.2 Client Architecture 15
4.3 Server Architecture 15
4.4 Data Collections 16
4.5 Repair Request Document 16
4.6 State Model 16
4.7 The Service Spine 18
4.8 API Design 18

5 Implementation **19**
5.1 Technology Stack 19
5.2 Development Process 20
5.3 Frontend Implementation 21
5.3.1 Role-Aware Routing 21
5.3.2 Server-State Hooks 22
5.3.3 Action-First Workspace 22
5.4 Backend Implementation 22
5.4.1 Controllers, Services and Adapters 22
5.4.2 Database Transactions 22
5.4.3 Notifications, Email and Scheduled Jobs 23

5.5 Service Catalogue and Matching 23
5.6 Pickup Scheduling 23
5.7 Quotation Model 23
5.8 Payment Implementation 24
5.9 Technician Settlement and Wallet 24
5.10 Damage and Evidence Images 25
5.11 Public Tracking and Jobs Board 25
5.12 Deployment and Continuous Integration 25
5.13 Final Improvement Phase 25

6 Core Workflows and Business Rules **27**
6.1 End-to-End Repair Workflow 27
6.2 Request Creation 27
6.3 Job Portal and Offers 27
6.4 Inspection Fee and Booking 29
6.5 Inspection, Quotation and Revision 30
6.6 Payment, Repair and Receipt 30
6.7 Settlement and Withdrawal Lifecycle 31
6.8 Cancellation Rules 32
6.9 Service-Area Map 32
6.10 Feedback and Moderation 32
6.11 Notifications and Reminders 32
6.12 Damage Image Upload 33

7 AI-Assisted Features **34**
7.1 AI Customer Support Chat 34
7.2 AI Request Helper 34
7.3 Technician Price Guide and AI Offer Note 35
7.4 AI Summary of Admin Reports 35
7.5 Safeguards 35

8 Security, Reliability and Data Integrity **37**

8.1 Security Principles . . . 37
8.2 Authentication . . . 37
8.3 Role- and State-Based Authorization . . . 37
8.4 Financial Integrity . . . 38
8.5 Privacy . . . 38
8.6 Private Image Security . . . 38
8.7 Input Validation and Safe Errors . . . 39
8.8 CORS, Rate Limiting and Abuse Control . . . 39
8.9 Transaction Safety . . . 39
8.10 Environment Isolation . . . 40
8.11 Legacy Compatibility Boundary . . . 40

9 Testing and Evaluation **41**

9.1 Testing Strategy . . . 41
9.2 Verification Results . . . 42
9.3 Representative Test Scenarios . . . 42
9.4 Manual End-to-End Acceptance . . . 44
9.5 Evaluation Against Objectives . . . 45
9.6 Validation Boundary . . . 45

10 Limitations and Future Work **46**

10.1 Current Limitations . . . 46
10.1.1 Manual Technician Payouts . . . 46
10.1.2 Card Payments in Test Mode . . . 46
10.1.3 Pickup Capacity Is Checked, Not Locked . . . 46
10.1.4 Provisional Catalogue Prices . . . 47
10.1.5 Single Active Job per Technician . . . 47
10.1.6 Legacy Internal Values . . . 47
10.1.7 Not Yet Used by Real Customers . . . 47
10.2 Future Enhancements . . . 47

11 Conclusion **49**

References **50**

Appendices **53**

A API Surface **54**

B Data Model Summary **57**

C Financial Rules **58**

D Source and Build Notes **59**

E Incident Record: Revised Quote Hang **61**

List of Tables

1.1 Project objectives 3

2.1 Comparison of approaches to electronics repair 8

3.1 User roles 9
3.2 Functional requirements 10
3.3 Non-functional requirements 12
3.4 Service catalogue summary with sample services 13

4.1 MongoDB collections 16
4.2 Repair lifecycle states 17
4.3 Representative API endpoints 18

5.1 Implemented technology stack 19
5.2 Development phases (from the git history) 20
5.3 Final improvement phase log 26

6.1 Inspection fee outcomes 29

7.1 AI risks and safeguards 36
7.2 Default AI usage limits 36

9.1 Verification results (11 October 2026) 42
9.2 Representative test scenarios 43
9.3 Evaluation against objectives 45

A.1 Sarabo API endpoints (main families) 54

B.1 Selected fields of a repair request 57

C.1 Worked example 58

D.1 Verified code baseline 59

List of Figures

3.1 Role-oriented use-case view 13

4.1 Sarabo high-level architecture 14
4.2 Conceptual structure of a V2 repair request 17
4.3 The four-stage service spine shown to users 18

6.1 End-to-end repair workflow 28
6.2 Rejected-quote recovery 30
6.3 Technician settlement and withdrawal lifecycle 31
6.4 Damage-image upload workflow 33

9.1 Testing layers used in Sarabo 41

Listings

5.1 Quote total and amount still to pay 23
5.2 Settlement rule (whole taka) 24
D.1 Development commands 59
E.1 Round-aware quote notification key 61

Chapter 1

Introduction

1.1 Project Overview

Sarabo is a web-based electronics repair service management platform developed as an individual final year project. It coordinates the complete life of a repair request (from the moment a customer describes a faulty device to the moment the technician is paid) through one traceable workflow. The platform connects three actors: a *Customer* who needs a repair, a *Technician* who performs it, and an *Admin* who keeps the service running.

The central design decision is that a repair request is a *stateful service case*, not a simple database record. Every important activity (request creation, technician offers, booking, pickup, inspection, quotation, payment, repair, handover, settlement and feedback) moves that case from one well-defined state to another, and the server checks that each move is allowed for that user at that moment. This is what separates Sarabo from a basic create-read-update-delete (CRUD) application: the correctness of a later action depends on earlier actions having happened, and having happened only once.

The project went through two major stages. The first stage (July–August 2026) built the core repair lifecycle with admin-driven technician assignment, and was described in an earlier version of this report. The second stage (September–October 2026) turned Sarabo into a two-sided marketplace: an open jobs board where technicians send offers, a paid inspection fee with automatic refunds, pickup scheduling, customer reviews, email notifications, operational reports, four AI-assisted features and a production deployment. This report describes the system as it stands after

both stages.

1.2 Problem Statement

A repair service involves both technical and administrative coordination. A customer needs to describe the device and problem, understand the likely cost before handing the device over, choose someone trustworthy, pay securely and know whether the device has been repaired. A technician needs a steady source of nearby jobs that match their skills, a structured way to record an inspection and quote, and a reliable way to be paid. An administrator needs to see which requests are stuck, which payments need attention, and whether demand is being turned away.

When these activities are handled through phone calls, messaging apps and paper records, inconsistent states easily occur: a customer is charged before agreeing to a price, a technician is booked twice for the same time, an inspection fee is neither refunded nor credited, a technician's earning is paid twice, or private photos of a customer's device are exposed. Sarabo addresses this by modelling the repair process as a role-aware workflow with explicit transition rules, server-side validation and transactional updates.

1.3 Motivation

The project was motivated by five goals:

1. to address a real, local problem: making electronics repair in Bangladesh more transparent and trustworthy for customers and more accessible to independent technicians;
2. to build a realistic full-stack system that goes beyond simple CRUD operations, where roles and state transitions are enforced on the server rather than trusted to the browser;
3. to integrate production-grade external services (identity, card payment and refunds, private object storage, email, generative AI) without exposing privileged credentials;
4. to apply generative AI where it genuinely helps users, while keeping it safe: no invented prices, no leaked personal data, and hard usage limits; and
5. to give every user a clear interface in which the next meaningful action is always visible.

1.4 Objectives

The main objective is to develop a reliable electronics repair management platform that manages the complete service lifecycle. The specific objectives are listed in Table 1.1.

Table 1.1: Project objectives

No.	Objective
O1	Provide secure authentication and role-aware access for Customer, Technician and Admin.
O2	Let customers create structured repair requests (device, service, region, pickup window, optional damage photos), with optional AI assistance.
O3	Match requests only to approved technicians with the right expertise in the customer's region, through an open offer-based job portal.
O4	Support a paid inspection fee that books a technician, is credited toward the final price, and is refunded or split fairly when the repair does not go ahead.
O5	Provide an inspection and quotation workflow with customer approval, rejection and revision.
O6	Accept payment only for approved amounts, with a single authoritative paid state and downloadable receipts.
O7	Support repair progress, completion evidence, receipt confirmation and customer feedback.
O8	Keep customers and technicians informed through in-app notifications, email, reminders and a sanitized public tracking page.
O9	Provide a technician wallet with a 90/10 settlement and controlled bKash withdrawals.
O10	Give admins operational oversight: attention lists, reports, catalogue management and moderation.
O11	Validate the system through automated server tests, browser tests, static checks and continuous integration, and deploy it.

1.5 Scope

The implemented scope includes registration and email verification, technician applications with structured expertise, a versioned service catalogue, AI-assisted request creation, damage-photo upload, pickup scheduling, an open jobs board with offers, inspection-fee booking and refunds, inspection, quotation and revision, Stripe payment, repair progress and completion evidence, receipt confirmation, reviews and private reports, in-app and email notifications, public tracking, technician wallet and bKash withdrawals, an AI customer-support chat, admin reports with an AI summary, and production deployment.

The project does not try to implement every capability of a commercial field-service company.

Automated mobile-wallet payouts, spare-parts inventory, warranty claims, live GPS tracking of technicians, native mobile apps and multi-branch accounting are outside the current scope and are discussed as future work in Chapter 10.

1.6 Individual Contribution

Sarabo was completed as a solo project by Jahid Hasan. The author was responsible for requirements analysis, data modelling, user-interface design, frontend and backend implementation, workflow and financial logic, integration of external services, AI prompt and safety design, testing, deployment, debugging and the preparation of this report.

1.7 Report Organization

Chapter 2 introduces the technical background and related systems. Chapter 3 defines stakeholders and requirements. Chapter 4 presents the architecture, data model and state model. Chapter 5 describes the implementation and the development process. Chapter 6 explains the core workflows and business rules. Chapter 7 covers the AI-assisted features and their safeguards. Chapter 8 discusses security, reliability and data integrity. Chapter 9 presents testing and evaluation. Chapter 10 lists limitations and future work, and Chapter 11 concludes the report. The appendices summarise the API, the data model, the financial rules, build notes and a record of notable incidents.

Chapter 2

Background and Related Work

2.1 Web-Based Service Management

A service management platform differs from a static information system because each request evolves through a sequence of operational decisions made by different people. Sarabo follows a resource-oriented HTTP API design influenced by the REST architectural style [1]. The browser presents the workflow, but the server remains the only authority on whether a transition is legal.

2.2 Single-Page Application Architecture

The client is a React single-page application. React provides a component model for composing stateful interfaces [2], React Router maps public pages and role-specific dashboard routes, and TanStack Query manages asynchronous server state, caching and invalidation [3]. This separation matters in Sarabo because server data such as request status, offers, quotations, wallet balances and notifications can change independently of the form a user is filling in. Vite [4] provides the development server and production bundling, and Tailwind CSS [5] the styling system.

2.3 RESTful API and Express

The server uses Express 5 [6] as its HTTP framework. Routes are grouped by domain (users, technicians, repair requests, job applications, inspections, quotations, payments, repairs, wallet,

feedback, notifications, tracking, service definitions, damage uploads, support and administration). Sensitive routes apply authentication middleware and role checks before any database operation.

2.4 Document Database and Transactions

MongoDB is the primary database [7]. A document model suits a repair request: it has a stable identity but accumulates nested workflow information (product snapshot, service and price snapshot, damage images, inspection, quote and quote history, payments, repair evidence, assignment history and settlement). Separate collections are used where an independent lifecycle, index or audit trail is needed, such as job applications, payments, notifications, tracking events, withdrawals and reviews. MongoDB multi-document transactions, available on replica sets, let several of these collections change together or not at all.

2.5 Authentication and Authorization

Firebase Authentication supplies user identity and signed ID tokens [8]. The client attaches the token to every protected API request, and the server verifies it with the Firebase Admin SDK before trusting the caller's email. Authorization is then role-based (Customer, Technician, Admin) and state-based (the request must be in the right stage). For backward compatibility the stored role value for Technician remains `rider`, inherited from the original codebase, while every user-facing label says "Technician".

2.6 Online Payment Processing

Sarabo uses Stripe Checkout for card payments [9]. The design treats the server-side record of payment as authoritative: Stripe calls a webhook whose signature is verified against a shared secret [10], so the payment is recorded even if the customer closes the browser before returning to the success page. Refunds are requested through Stripe's API with idempotency keys so they cannot be issued twice. All payments currently run in Stripe test mode.

2.7 Private Object Storage

Damage and repair-evidence images are stored in a private Supabase Storage bucket [11]. The server issues short-lived signed URLs: the browser uploads bytes directly to storage, the server then verifies the stored object's real type and size, and every later view uses a fresh signed read URL. Large files never pass through the API server, and the privileged storage key never reaches the browser.

2.8 Large Language Models in Applications

Large language models (LLMs) such as Google Gemini [12] can turn free text into structured data, summarise information and draft natural-language messages. They can also produce confident but wrong output ("hallucination"), be manipulated by instructions hidden in user text (prompt injection), and generate unpredictable cost. OWASP lists prompt injection and overreliance among the main risks of LLM applications [13]. Sarabo therefore uses AI only as an *assistant*: the model's answer is constrained to a JSON schema, validated against live data on the server, never allowed to set a price, and always reviewed by a human before it takes effect (Chapter 7).

2.9 Usability and Accessibility

The interface follows an action-first approach: the current stage of a repair and the next action for the signed-in role get more visual weight than secondary data. A recurring "service spine" (Request, Inspect, Approve, Repaired) appears on the home page, the public tracking page and every dashboard. Labels, focus order, keyboard access, colour contrast, reduced motion and responsive layouts from 320 px upward follow the WCAG 2.2 guidance [14].

2.10 Related Systems

Repair and home-service needs in Bangladesh are served today in three broad ways:

- **Local repair shops and freelance technicians**, found through word of mouth or social media. Prices are negotiated in person, there is no tracking and little recourse.

- **Brand service centres**, which offer reliable repair for their own products but cover only one brand and are concentrated in large cities.
- **General on-demand service marketplaces** such as Sheba.xyz in Bangladesh [15] and Urban Company in South Asia [16], which let customers book many kinds of household services from listed providers.

International platforms such as Thumbtack [17] popularised the model in which customers describe a job and several professionals respond with quotes. Sarabo adopts this offer model but specialises it for electronics repair, where the true price is usually unknown until the device has been opened. Table 2.1 compares the approaches on the features most relevant to this project.

Table 2.1: Comparison of approaches to electronics repair

Capability	**Local shop**	**General marketplace**	**Sarabo**
Online request with device and problem details	–	✓	✓
Price estimate before hand-over	–	Partial	✓
Several competing offers per job	–	Varies	✓
Separate inspection fee credited to final price	–	–	✓
Formal inspection and itemised quote	–	Varies	✓
Online payment with receipt	–	✓	✓
Step-by-step public tracking	–	Partial	✓
Technician matched by repair-specific expertise	–	Partial	✓
Bangla/Banglish AI request assistance	–	–	✓

Sarabo's contribution is not a new algorithm. Its engineering value lies in combining patterns from ticketing systems (a trackable case with an identifier and history), field-service systems (assignment, inspection, completion), marketplaces (offers, payment, commission, payouts) and evidence-driven workflows (private photos and completion records) into one coherent repair workflow with explicit state, money and authorization rules.

Chapter 3

Requirements Analysis

3.1 Stakeholders and Roles

The system has three signed-in roles and one anonymous audience. Their goals are summarised in Table 3.1.

Table 3.1: User roles

Role	Primary goal	Representative actions
Customer	Get a device repaired at a fair, known price	Create request (optionally with AI help), add photos, pick a pickup slot, compare offers, pay inspection fee, approve or reject quote, pay, track, confirm receipt, review or report the technician
Technician	Find suitable work and get paid	Apply with expertise and region, browse the jobs board, send offers, inspect, quote, revise, repair, complete, view wallet, withdraw via bKash
Admin	Keep the service healthy	Approve technicians, edit catalogue and prices, handle missed pickups and refunds, process withdrawals, moderate reviews and reports, read reports
Visitor	Learn about the service	Browse services, service areas and open jobs, track a repair with a tracking code, ask the AI support chat

3.2 Functional Requirements

Table 3.2 lists the functional requirements. Requirements FR-21 to FR-32 were added in the second development stage.

Table 3.2: Functional requirements

ID	Requirement
FR-01	The system shall authenticate users (email/password and Google) and protect private routes.
FR-02	The system shall distinguish Customer, Technician and Admin capabilities on the server.
FR-03	A verified Customer shall be able to create a structured repair request for a catalogue service.
FR-04	The service catalogue shall provide active device and repair categories with BDT estimate ranges, inspection fee caps and required expertise level.
FR-05	Customers shall be able to attach up to three damage photos through a controlled private-storage workflow.
FR-06	Users shall be able to apply as Technicians with region, district and structured expertise; Admin shall approve or reject applications.
FR-07	The assigned Technician shall submit an inspection before a quote can be submitted.
FR-08	The Technician shall submit an itemised quote (labour, parts, additional charges).
FR-09	The Customer shall approve or reject the quote; after rejection the Technician may revise once more or close the request.
FR-10	Payment shall be possible only for an approved quote and never twice.
FR-11	The Technician shall record repair progress and completion with optional evidence photos.
FR-12	The Customer shall confirm receipt of the repaired device.
FR-13	The system shall send role-appropriate in-app notifications for workflow events.
FR-14	A public tracking page shall show a sanitized timeline by tracking code, without personal or payment data.
FR-15	A 90/10 Technician/platform settlement shall be frozen when the repair is paid.
FR-16	Technicians shall request withdrawals up to their server-computed available balance; Admin shall mark each as paid (with bKash TrxID) or rejected exactly once.
FR-17	A customer shall be able to cancel a request before a technician is booked.

ID	Requirement
FR-18	Admin shall be able to edit the service catalogue; price changes shall create a new version without changing existing requests.
FR-19	Technicians shall be able to edit their expertise; Admin shall see technician availability.
FR-20	Customers shall be able to download a PDF receipt for every payment.
FR-21	A request shall be accepted only if at least one approved Technician in the customer's region can perform it; refused demand shall be counted.
FR-22	Customers shall choose a two-hour pickup slot (Saturday–Thursday, up to 7 days ahead) subject to per-region capacity, and may reschedule before pickup.
FR-23	Open requests shall appear on a public jobs board; matching Technicians shall be notified and may send offers (estimate range, inspection fee, note).
FR-24	The Customer shall compare offers and choose one, or reject one (the job is reposted and the Technician may send one revised offer).
FR-25	Choosing an offer with a fee shall open a Stripe payment; the confirmed payment shall book the Technician atomically.
FR-26	The inspection fee shall be credited toward the final quote, refunded when the visit fails or is cancelled in time, and split 90/10 when kept.
FR-27	Customers shall rate a completed repair (1–5 with optional comment) or file a private report; Admin shall moderate both.
FR-28	The system shall email request confirmations, pickup reminders and new-job alerts, and post in-app pickup reminders daily.
FR-29	Admin shall see an attention list (overdue and missed pickups, unmatchable requests, no offers in 24 hours, refunds waiting) and period reports.
FR-30	An AI helper shall turn a free-text description and/or damage photo into a draft request that the customer reviews.
FR-31	Technicians shall see a price guide from past offers and may ask AI to draft their offer note.

ID	Requirement
FR-32	An AI support chat shall answer visitor questions from live catalogue data, and Admin reports shall include an AI summary checked against the real figures.

3.3 Non-Functional Requirements

Table 3.3: Non-functional requirements

Category	Requirement
Security	Identity, role, prices, fees, balances and refund decisions are derived or validated on the server. Private images are reachable only through short-lived signed URLs. Secrets never reach the browser.
Consistency	Steps that change several records (booking, approval, completion, settlement, refunds) run as database transactions.
Reliability	Repeated or concurrent actions return controlled conflicts instead of repeating money or state changes. Failed refunds stay pending and can be retried. Email or notification failures never fail the main action.
Privacy	Public pages (tracking, jobs board) expose no names, emails, addresses or payment data. AI providers receive no personal identifiers.
Cost control	Every AI feature has per-user and site-wide daily limits stored in the database, and is off unless explicitly enabled.
Usability	Each screen shows the current stage and the next action for that role; empty, loading and error states are truthful.
Accessibility	Keyboard access, visible focus, sufficient contrast, labelled controls, reduced-motion support, layouts from 320 px.
Performance	Cached server state on the client, code-split routes, direct-to-storage uploads, paginated lists.
Maintainability	Workflow rules, status presentation and query keys are centralised; CI runs lint, build and tests on every push.
Auditability	Tracking events, assignment history, quote history, payment rows, refund state, withdrawal processing and moderation history are kept.

3.4 Service Catalogue

The catalogue contains **45 services** across eight device categories (Table 3.4). Each service pairs a device category with a repair category and carries a BDT estimate range, an inspection-fee cap, a required expertise level (beginner, intermediate, advanced, expert) and an estimated duration. Prices are shown as estimates; the final amount is always the technician's written quote

after inspection.

Table 3.4: Service catalogue summary with sample services

Device	Services	Sample service	Estimate (BDT)
Smartphone	8	Screen Replacement	1,500–6,000
Laptop / Computer	8	Motherboard Repair	2,500–12,000
Television	5	Screen / Panel Repair	4,000–18,000
Refrigerator	6	Compressor / Refrigerant Service	3,500–15,000
Washing Machine	6	Control Board Repair	2,000–7,000
Air Conditioner	6	Gas Refill / Compressor Repair	3,000–20,000
Microwave Oven	4	Electrical / Power Repair	1,500–5,000
Other Electronics	2	General Repair	1,000–5,000
Total	**45**		

3.5 Use-Case View

Figure 3.1 shows the main interactions of each role with the platform.

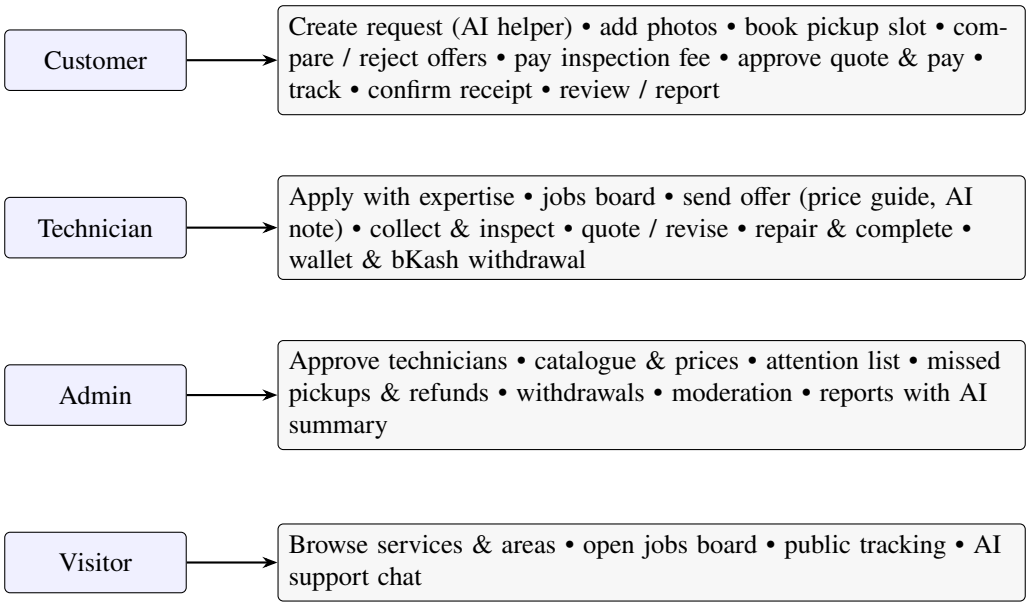


Figure 3.1: Role-oriented use-case view

Chapter 4

System Design and Architecture

4.1 High-Level Architecture

Sarabo follows a browser-client / API-server architecture with external services for identity, payment, storage, email and AI (Figure 4.1). The React client is hosted on Firebase Hosting; the Express API runs as a serverless function on Vercel, which also triggers a daily scheduled job. MongoDB Atlas holds all application data.

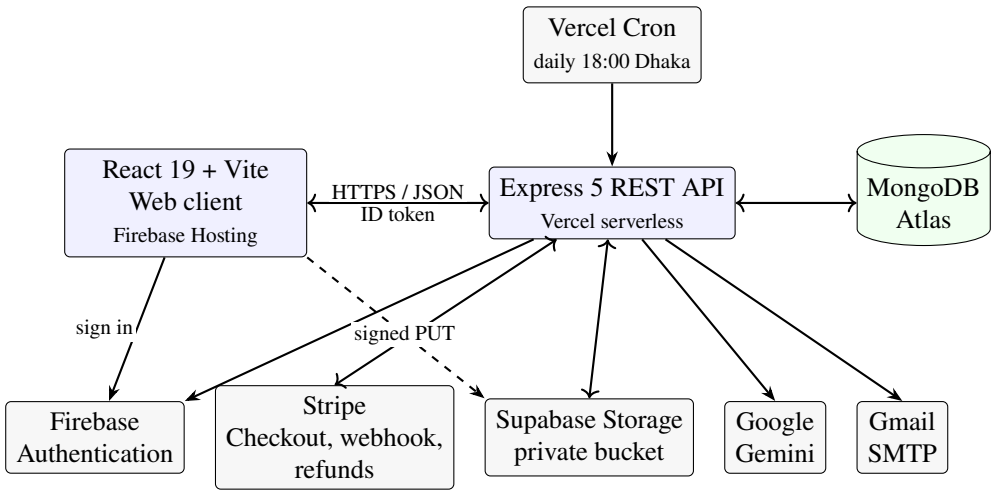


Figure 4.1: Sarabo high-level architecture

4.2 Client Architecture

The client (about 337 source files) is organised around five concerns:

1. **Routing and guards**: public pages (home, services, service areas, jobs board, tracking, about, become a technician), authentication pages, and a dashboard subdivided by `CustomerRoute`, `TechnicianRoute` and `AdminRoute`. Guards improve usability only; the server enforces every rule again.

2. **Feature components**: request creation and the AI helper, damage images, pickup slots, job offers, inspection, quotation, payment, repair, feedback, wallet, notifications, support chat and administration.

3. **Server-state hooks**: TanStack Query hooks with central query-key factories provide caching, loading and error states, mutations and targeted invalidation.

4. **Presentation utilities**: status-to-stage mapping (the "service spine"), currency formatting, eligibility helpers and per-action error messages.

5. **Design system**: shared tokens (colours, type scale, radii), light and dark themes, and reusable primitives built on Radix UI.

4.3 Server Architecture

The server (about 191 JavaScript files including tests) is layered as *routes* → *middleware* → *controllers* → *services* / *utilities* → *database*. Middleware attaches a request ID, logs safely, verifies the Firebase token and waits for the database connection. Controllers own the state rules of one domain; services hold logic shared by several endpoints (payment eligibility, job booking, refunds, settlement, notifications, matching, AI calls, email). External providers (Stripe, Supabase, Gemini, Gmail) sit behind small adapters, so automated tests can inject fakes and never contact the real services.

The Stripe webhook route is registered before the global JSON parser because signature verification needs the exact raw request body. Routes stay registered even if the database is briefly unavailable; database-backed requests then fail with a controlled 503 instead of a misleading 404.

4.4 Data Collections

Table 4.1 lists the 19 MongoDB collections.

Table 4.1: MongoDB collections

Collection	Purpose
`users`	Accounts and the authoritative role.
`technicians`	Application, approval status, expertise, region/district, work status, bKash details.
`repair_requests`	The repair case with all lifecycle sub-documents.
`job_applications`	Technician offers on open requests and their states.
`service_definitions`	Versioned catalogue of services and prices.
`payments`	Stripe payment rows for inspection fees and repairs, including refund state.
`checkout_sessions`	Checkout idempotency and expiry.
`technician_withdrawals`	Withdrawal requests and admin processing.
`technician_reviews`	Customer ratings with moderation history.
`technician_reports`	Private customer reports with status and admin notes.
`notifications`	In-app notifications with de-duplication keys.
`tracking_events`	Sanitized public timeline.
`damage_upload_sessions`	Customer photo upload sessions.
`repair_evidence_sessions`	Technician completion-evidence sessions.
`deletion_cleanups`	Retry records for storage clean-up after deletion.
`turned_away_requests`	Hashed count of refused demand (180-day expiry).
`email_log`	Sent/failed email record (90-day expiry).
`support_rate_limits`	Shared AI rate-limit counters (auto-expiring).
`ai_report_summaries`	Last AI summary per report period.

4.5 Repair Request Document

Modern requests carry `schemaVersion: 2`. Device identity is a structured `product` snapshot (category, brand, model) and the chosen service is copied together with its price rule, so later catalogue edits never change an existing request. Figure 4.2 shows the main parts.

4.6 State Model

The stored status field `deliveryStatus` and several of its values were inherited from the original parcel-delivery codebase. They are kept as a frozen internal contract, while all user-facing labels use repair terminology (Table 4.2).

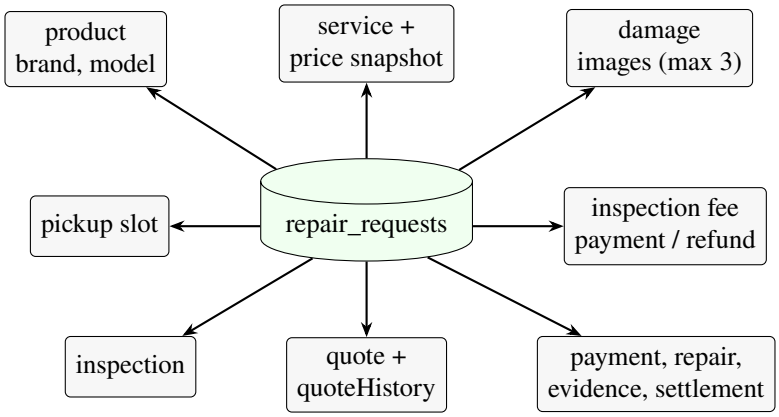


Figure 4.2: Conceptual structure of a V2 repair request

Table 4.2: Repair lifecycle states

Stored state	Meaning shown to users
`pending-pickup`	Waiting for a technician; open on the jobs board.
`assignment_pending`	Offered directly by an admin; technician to accept or reject.
`driver_assigned`	Technician booked.
`rider_arriving`	Technician on the way.
`parcel_picked_up`	Device collected.
`inspection_completed`	Inspection submitted.
`quote_submitted`	Final price waiting for the customer.
`quote_approved`	Customer accepted the price.
`quote_rejected`	Customer declined the price.
`payment_completed`	Customer paid.
`repair_in_progress`	Repair underway.
`repair_completed`	Repair finished; waiting for hand-over.
`parcel_delivered`	Customer confirmed receipt (terminal).
`cancelled`	Ended through an allowed path.

A job application (offer) has its own states: `pending`, `accepted`, `not_chosen`, `withdrawn`, `no_show` and `declined`. A settlement moves from *pending* to *available*, and a withdrawal from *requested* to *paid* or *rejected*.

4.7 The Service Spine

The interface compresses the detailed states into four stages that customers understand (Figure 4.3). The same component appears horizontally on the home page and vertically on the tracking page and in the dashboards. Receipt confirmation is shown as a hand-over action after "Repaired" rather than as a fifth stage.

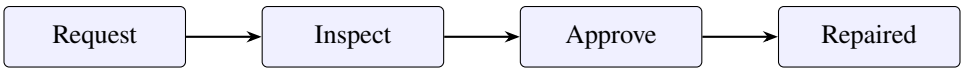


Figure 4.3: The four-stage service spine shown to users

4.8 API Design

The API exposes around 100 endpoints grouped into resource families. Table 4.3 lists representative ones; Appendix A gives a fuller list.

Table 4.3: Representative API endpoints

Method	Route	Purpose
POST	`/repair-requests`	Create a request
GET	`/jobs`	Public open-jobs board
POST	`/repair-requests/:id/applications`	Technician sends an offer
POST	`/repair-requests/:id/applications/:aid/accept`	Customer chooses an offer
POST	`/repair-requests/:id/inspection`	Submit inspection
POST	`/repair-requests/:id/quote/decision`	Approve or reject quote
POST	`/repair-requests/:id/checkout-session`	Start repair payment
POST	`/stripe-webhook`	Signed payment events
POST	`/repair-requests/:id/repair/complete`	Complete repair
POST	`/repair-requests/:id/review`	Rate the technician
POST	`/ai/request-draft`	AI request helper
GET	`/admin/attention`	Items needing admin action
GET	`/public/trackings/:code`	Sanitized tracking

Chapter 5

Implementation

5.1 Technology Stack

Table 5.1: Implemented technology stack

Layer	Technology	Role in Sarabo
Frontend	React 19, Vite 7	Component-based single-page app and build tooling
Routing	React Router 7	Public, authenticated and role-specific routes
Server state	TanStack Query 5, TanStack Table	Caching, mutations, invalidation; admin data tables
UI	Tailwind CSS 4, Radix UI, Motion, Recharts, Leaflet	Design system, accessible primitives, animation, charts, service-area map
Forms	react-hook-form	Validated multi-step forms
Backend	Node.js, Express 5	REST API, middleware, controllers, services
Database	MongoDB 7 driver, MongoDB Atlas	Persistence and multi-document transactions
Identity	Firebase Authentication + Admin SDK	Sign-in (email, Google) and token verification
Payments	Stripe 20	Checkout, signed webhooks, refunds
Storage	Supabase Storage	Private images with signed URLs
AI	Google Gemini (`@google/genai`)	Support chat, request helper, offer note, report summary
Email	Nodemailer (Gmail)	Request, reminder and job-offer emails
Documents	pdfkit	PDF payment receipts
Testing	Custom Node test harness, Playwright	Server integration tests; browser tests
Delivery	GitHub Actions, Vercel, Firebase Hosting	CI, serverless API, cron, static hosting

5.2 Development Process

Sarabo was developed iteratively in small, reviewable phases between 26 July and 4 October 2026, followed by a final improvement phase up to 11 October: 156 commits in the client repository and 107 in the server repository. Each phase followed the same loop: define the rule, implement it on the server with tests, build the interface, run lint, build and tests, then review the result before moving on. Larger features were developed on separate branches and merged through pull requests once continuous integration passed. Table 5.2 summarises the phases.

Table 5.2: Development phases (from the git history)

Phase	Dates	Main work
1. Foundation and hardening	26–28 Jul	Imported the base codebase; secured authentication and role routing; secure Stripe checkout and webhook; validated status transitions; public sanitized tracking; transactional assignment, approval and completion.
2. Admin and notifications	29 Jul – 4 Aug	Admin request management; in-app notifications; production database isolation; health monitoring; double-booking prevention.
3. Repair domain (V2)	5–8 Aug	Service taxonomy and priced catalogue; technician expertise; V2 request schema; expertise-based matching; damage uploads; prices in BDT; inspection, quote, payment and repair workflows; safe deletion; verified email.
4. Rename to Sarabo	10–12 Aug	Migrated database, collections, fields, modules, API routes and UI from parcel to repair vocabulary; assignment accept/reject; receipt confirmation.
5. Money and UI redesign	13–19 Aug	90/10 settlement; wallet and withdrawals; storage moved to Supabase; complete "service spine" redesign; accessibility pass. *(Earlier report submitted 20 Aug.)*

Phase	Dates	Main work
6. AI support and feedback	18–24 Sep	Gemini support chat; reviews and private reports with moderation; product tour; second redesign; admin catalogue editor; CI on GitHub Actions.
7. Operations	24–25 Sep	Pickup scheduling; region rule; email updates; Playwright tests; admin reports and attention list; in-app reminders; refusing unmatchable requests.
8. Job portal and inspection fee	25 Sep – 2 Oct	Technician offers and customer choice; inspection-fee payment, credit, refunds and split; reject-and-repost; public jobs board; job notifications; PDF receipts; one-step approve-and-pay.
9. AI helpers and payouts	2–4 Oct	AI request helper (text and photo); price guide and AI offer note; AI report summary; bKash withdrawals; hosting configuration.
10. Final improvements	from 11 Oct	Fixes and improvements for the final submission, recorded step by step in Section 5.13.

5.3 Frontend Implementation

5.3.1 Role-Aware Routing

The dashboard is wrapped by an authenticated route and subdivided by role guards. Customers get Overview, Create Request, My Requests and Payment History; Technicians get Overview, Find jobs, Assigned jobs, Completed jobs and Wallet; Admins get Overview, Repair requests, Assign technicians, Technicians, Withdrawals, Users, Services & prices, Reports, Technician reports and Technician reviews. Every role also has Notifications and Profile, and a bottom tab bar replaces the sidebar on phones. Deep links (for example `?category=smartphone`) survive the sign-in redirect. Routing is a convenience only; every protected endpoint re-checks authentication and authorization.

5.3.2 Server-State Hooks

Each feature has a query-key factory (for example `quoteKeys`, `repairKeys`, `walletKeys`) and hooks for reading and mutating. A mutation invalidates exactly the queries that become stale: approving a quote refreshes the quote, the request detail, the customer's request list and the payment eligibility. This keeps the three roles' views consistent without reloading the page.

5.3.3 Action-First Workspace

Each request page shows the service spine at the top, then a "next step" panel that tells the signed-in role exactly what to do (for example "Choose a technician: 2 offers waiting" or "Approve and pay BDT 2,300"), followed by supporting context. Reusable empty, loading, error, confirmation and form components keep behaviour consistent. A guided product tour introduces each dashboard on first visit, and a command menu allows keyboard navigation.

5.4 Backend Implementation

5.4.1 Controllers, Services and Adapters

Workflow endpoints live in domain controllers (for example `jobPortalController`, `quoteController`, `paymentController`). Logic used in more than one place lives in services such as `jobBooking`, `inspectionFeeRefunds`, `paymentEligibility`, `technicianMatchGate`, `notificationService` and `emailService`. External providers are reached through adapters (`supabaseBucketAdapter`, Gemini providers, Stripe client), which tests replace with fakes.

5.4.2 Database Transactions

Operations that update several dependent records use MongoDB transactions. Booking a technician after an inspection-fee payment, for instance, records the payment, sets the request to `driver_assigned`, marks the technician busy, accepts the chosen offer and marks every other offer `not_chosen`, all in one transaction (`services/jobBooking.js`). If any step fails, nothing changes.

5.4.3 Notifications, Email and Scheduled Jobs

Important events create in-app notifications through one service, each with a de-duplication key so that a retried request never notifies twice. Emails (request received, pickup reminder, new job offer) are sent through Gmail after the main action succeeds; a failed email is logged in `email_log` and never fails the request. A daily Vercel Cron job, protected by a secret, posts "pickup tomorrow" reminders to customers and technicians and automatically closes requests whose declined quote has had no response for three days.

5.5 Service Catalogue and Matching

Admins edit services and prices from the dashboard. A price change increments `pricingRule.version`; existing requests keep their own price snapshot. When a customer creates a request, `technicianMatchGate` checks that at least one approved technician in the same region has the required device, repair and expertise level. If not, the API answers `409 NO_TECHNICIAN_AVAILABLE`, the form explains why, and the refused demand is counted once per customer, service, region and day (with the customer identity hashed) so admins can see where to recruit.

5.6 Pickup Scheduling

Customers pick one of four two-hour windows (10:00–18:00 Bangladesh time), Saturday to Thursday, at least one hour and at most seven days ahead. Each region accepts a configurable number of pickups per slot (default three); a full slot returns `409 PICKUP_SLOT_FULL`. The window can be changed until the device is collected, and a moved pickup is reminded again.

5.7 Quotation Model

A quote has whole-taka labour, parts and additional-charge amounts, a total, currency, version and status. If the customer rejects it, the rejected quote is moved to `quoteHistory` before the live quote is cleared, so the old decision stays auditable and can never be paid by mistake.

Listing 5.1: Quote total and amount still to pay

```
repairSubtotal = laborAmount + partsAmount + additionalCharges
quote.totalAmount = repairSubtotal (currency "BDT")
amountDue = max(quote.totalAmount - inspectionFeePaid, 0)
```

5.8 Payment Implementation

The server creates every Stripe Checkout session from stored data (the approved quote or the chosen offer's fee), never from an amount sent by the browser. Each session carries metadata saying what it pays for (`repair` or `inspection_fee`). The signed webhook, and the success page as a fallback, both call the same idempotent processor, so a payment is recorded exactly once whichever arrives first. One shared server function decides whether a request is paid, and the same rule is used for payment eligibility, admin filters and list badges, so they can never disagree. Every payment row produces a PDF receipt (Stripe payment ID, date in Dhaka time, device, repair, technician, amount and refund status).

5.9 Technician Settlement and Wallet

When a repair is paid, a settlement snapshot is frozen from the approved quote:

Listing 5.2: Settlement rule (whole taka)

```
commission = round(repairSubtotal * 10 / 100) # Sarabo
receivable = repairSubtotal - commission # Technician
# invariant: commission + receivable == repairSubtotal
```

The settlement stays *pending* until the customer confirms receipt, then becomes *available*. Wallet balances are never stored as a running number; they are recomputed from settlement snapshots and withdrawal records, so they cannot drift. A technician requests a withdrawal up to the available balance, giving a bKash number; an admin sends the money with bKash "Send Money" and records the transaction ID (TrxID) when marking it paid. A unique partial index allows only one open withdrawal per technician.

5.10 Damage and Evidence Images

Images go directly from the browser to a private Supabase bucket through a signed upload URL (JPEG, PNG or WebP, up to 8 MB, at most three per request). The server then reads the stored object's real type and size before attaching it, and returns fresh five-minute signed read URLs only to the owner, an admin or the currently assigned technician. When a request is deleted, storage clean-up is retried in the background until it succeeds.

5.11 Public Tracking and Jobs Board

New requests get a tracking code `SRB-<random>` with 128 bits of randomness. `GET /public/trackings/:code` returns only the status timeline and timestamps, is rate limited and accepts only exact codes. The public jobs board (`GET /jobs`, 60 requests per minute per address) lists open requests with device, repair, area, pickup day, usual price and number of offers, but never the problem text, photos, tracking code or any personal detail.

5.12 Deployment and Continuous Integration

The API is deployed as a Vercel serverless function (Node.js 24) and the client on Firebase Hosting at `https://sarabo-jahid.web.app`. Production uses its own database name, and the server refuses to start in production without the required secrets and an `https` client origin. GitHub Actions runs on every push: for the server, the full test suites against a throwaway MongoDB replica set started inside the job (CI is never given the real database credentials); for the client, ESLint and a production build. Deployment is a deliberate manual step.

5.13 Final Improvement Phase

After the system was feature-complete on 4 October 2026, a final phase of fixes and improvements began on 11 October 2026 to prepare the project for submission. It started with a full review of both codebases and their documentation and a fresh verification run of every automated check (Chapter 9). Each later change is recorded with its problem, change, files and verification in the project work log and summarised in Table 5.3.

Table 5.3: Final improvement phase log

#	Date	Change and verification
0	11 Oct 2026	Codebase and documentation review; re-ran server suites against an isolated in-memory MongoDB replica set, client lint, production build and Playwright browser tests.
1	11 Oct 2026	Service-area map: OpenStreetMap coordinates for 226 of the 236 covered sub-areas, each kept only if it lies inside its own district. Searching a sub-area (e.g. Ambarkhana) now zooms to that spot instead of the district centre. Verified with two new browser tests on desktop and mobile.
2	11 Oct 2026	Final payout on technician removal: removing a technician queues their whole available balance as a bKash withdrawal in the same transaction (Section 6.7). Verified with 15 new server checks.
3	11 Oct 2026	Homepage hero: the photo moved to the right side (top band on phones) so the headline no longer sits over it. Checked with screenshots at three screen widths.
4	11 Oct 2026	Final verification of the committed code and CI on both repositories (Section 9.2).

Chapter 6

Core Workflows and Business Rules

6.1 End-to-End Repair Workflow

Figure 6.1 shows the main path of a repair, from request to settlement.

6.2 Request Creation

The customer chooses a device category and a repair service from the live catalogue (or lets the AI helper propose them), adds brand, model and a description, optionally adds up to three photos, and picks a pickup window. The estimate card shows the catalogue's price range. Before submitting, the form calls `GET /service-availability`; the server repeats the check on submission and refuses the request with a clear explanation if no approved technician in that region can do this repair at the required level. Accepted requests receive a tracking code and a confirmation email.

6.3 Job Portal and Offers

A request is *open* while it is waiting for a technician. It appears on the public jobs board as a safe summary, and every approved technician who matches it receives a `job_posted` notification (at most once per request). A technician who starts matching later, after being approved or adding a skill, is told about the open jobs they can now take. Approved technicians see the details they need (district, pickup time, problem description and photos) but never the customer's name, email or street address.

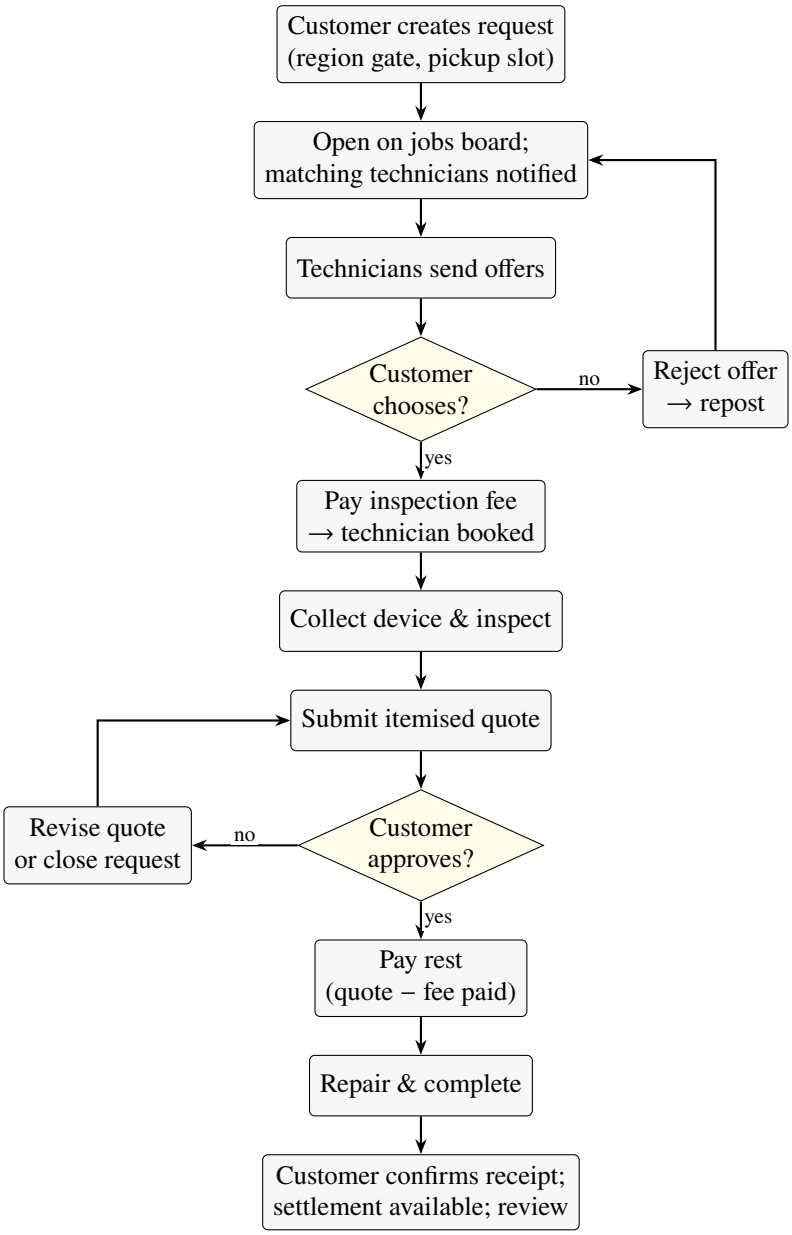


Figure 6.1: End-to-end repair workflow

A technician sends an *offer*: an estimate range, an inspection fee (capped by the catalogue fee for that service) and a note. The customer compares offers side by side and either chooses one or rejects one with a reason (price, inspection fee or other). A rejected offer causes the job to be *reposted* to the top of the board; the rejected technician may send one revised offer, and other matching technicians are notified. If no offer is waiting 24 hours after the job was last posted, it appears on the admin attention list. Admins can also invite a specific technician to apply, or assign one directly as an override.

6.4 Inspection Fee and Booking

Choosing an offer with a fee opens a Stripe Checkout for exactly that fee; nothing is booked yet. When Stripe confirms the payment, one transaction records it and books the technician. If the technician can no longer be booked at that moment (busy, request cancelled, someone else chosen), the payment is recorded and refunded automatically and the customer is asked to choose again. The fee is stored separately from the repair payment and is later credited toward the final price. Table 6.1 shows what happens to the fee when the repair does not go ahead.

Table 6.1: Inspection fee outcomes

Situation	Fee
Repair approved and paid	Credited toward the final price
Technician does not come (admin records a missed pickup)	Full refund; other offers can be chosen
Customer cancels 2 hours or more before the pickup window	Full refund
Customer cancels less than 2 hours before	Kept: 90% technician, 10% Sarabo
Customer declines the final price and the request ends	Kept: 90% technician, 10% Sarabo
Booking fails after payment	Automatic full refund

Refunds happen in two steps. Inside the transaction that causes the refund, the fee and its payment row are marked *refund pending*. After the transaction commits, Stripe is asked to refund with the idempotency key `inspection-refund-<session id>`, which guarantees at most one refund. If Stripe fails, the refund stays pending, appears on the admin home page, and can be retried with one click; it is never silently lost.

6.5 Inspection, Quotation and Revision

After collecting the device, only the booked technician can submit the inspection, which records findings and detected issues. The quote can be submitted only from `inspection_completed`. The customer sees the quote together with the fee already paid and the amount still due, and the approve button reads "Approve and pay BDT *X*", so approval leads straight to payment.

If the customer rejects the quote, the technician may revise or close the request (Figure 6.2). Revising archives the old quote and reuses the normal submit-and-decide cycle. If the technician neither revises nor closes within three days, the daily job closes the request automatically; the request records who closed it (`technician`, `customer` or `auto`), and the inspection fee is kept and split.

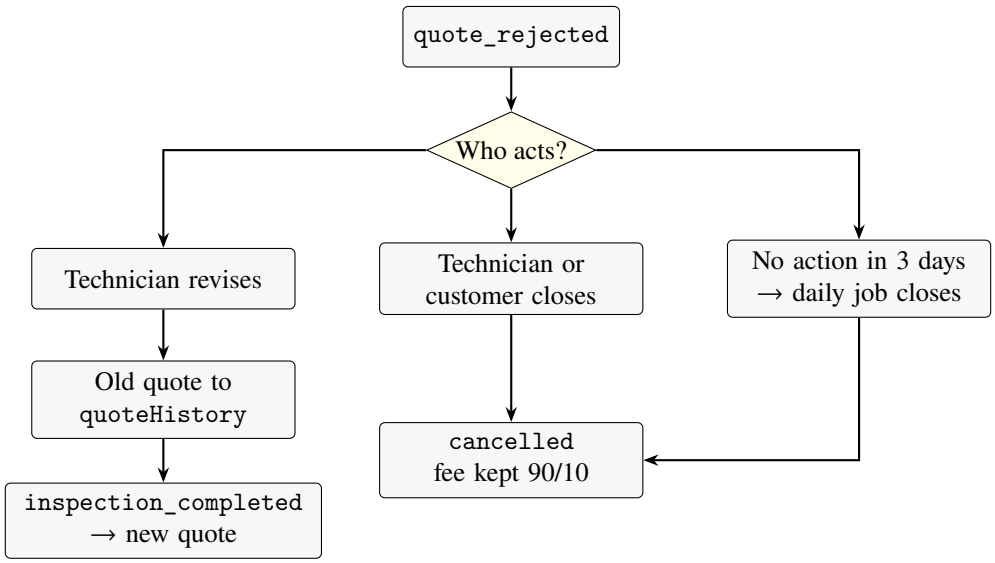


Figure 6.2: Rejected-quote recovery

Each quote round has its own notification identity, derived on the server as `quoteHistory.length + 1`. A retried submission of the same round is therefore harmless, while a genuine revision produces a new notification (see Appendix E).

6.6 Payment, Repair and Receipt

Payment is possible only for an approved quote that is not yet paid. After payment the technician starts the repair, adds progress updates and completes it, optionally with evidence photos. Com-

pletion releases the technician for other work and shows the customer a “Confirm receipt” action. Only that confirmation moves the request to its terminal state and makes the technician’s earning available, so a technician cannot withdraw money for a device the customer has not received.

6.7 Settlement and Withdrawal Lifecycle

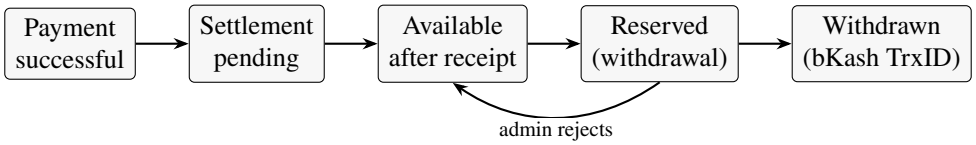


Figure 6.3: Technician settlement and withdrawal lifecycle

The wallet is computed, not stored:

$$\text{Pending} = \sum \text{paid settlements not yet receipt-confirmed} \tag{6.1}$$

$$\text{Gross available} = \sum \text{receipt-confirmed settlements} + \sum \text{kept inspection-fee shares} \tag{6.2}$$

$$\text{Reserved} = \sum \text{open withdrawal requests} \tag{6.3}$$

$$\text{Withdrawn} = \sum \text{paid withdrawals} \tag{6.4}$$

$$\text{Available} = \text{Gross available} - \text{Reserved} - \text{Withdrawn} \tag{6.5}$$

Because rejection simply removes a reservation, no compensating balance write is ever needed.

Final payout when a technician is removed. When an admin removes an approved technician, their whole available balance is queued as a withdrawal to their bKash number in the same database transaction as the removal, so either both happen or neither does. The number is taken, in order, from the admin’s entry in the removal dialog, an already-open request (which is topped up rather than duplicated), the technician’s last withdrawal, or their application phone. If money is owed and no valid number exists, the removal is refused until the admin enters one. Pending money is not yet available, so it is not included. The payout is tagged as a final payout in the admin queue and is sent and recorded with a bKash TrxID like any other withdrawal. Sarabo has no bKash API, so the transfer itself remains a manual bKash Send Money.

6.8 Cancellation Rules

Before a technician is booked, a customer can cancel a request freely; a cancelled request can be requested again with its details pre-filled. After booking with a paid fee, `GET /repair-requests/:id/booking-cancellation` tells the customer in advance which outcome cancelling now would have (refund or kept fee), and the cancellation itself is allowed only until the device is collected.

6.9 Service-Area Map

The public Service Areas page draws every listed district on an OpenStreetMap map (Leaflet) from the same `serviceAreas.json` file that drives the request form. Each district also stores the coordinates of its covered sub-areas. These were looked up once, offline, from OpenStreetMap and kept only if they lie inside that district (226 of 236). A search lists matching sub-areas before districts, so "Ambarkhana" opens Ambarkhana in Sylhet at street level instead of the city centre. A sub-area without a stored point falls back to its district's pin, and nothing is geocoded in the browser.

6.10 Feedback and Moderation

After a completed and receipt-confirmed repair, the customer may leave one review: a rating from 1 to 5 and an optional comment (up to 1,000 characters, plain text). Reviews are visible by default and admins can hide or restore them with a reason; every change is versioned and kept in a moderation history. Separately, a customer can file a private report about a technician (for example unprofessional behaviour, device damage or a payment issue). Reports go only to admins, move through *open*, *under review*, *resolved* or *dismissed*, and are never deleted with the request. Technicians see their average rating and visible reviews, but never reports or the reviewer's identity.

6.11 Notifications and Reminders

Forty-one notification types, all defined in one module (`utils/notificationEvents.js`), cover offers, booking, refunds, inspection, quotes, payment, repair milestones, reviews, with-

drawals and a removed technician's final payout. Each has a de-duplication key. The daily job sends "pickup tomorrow" reminders in the app (and by email when enabled), once per pickup slot.

6.12 Damage Image Upload

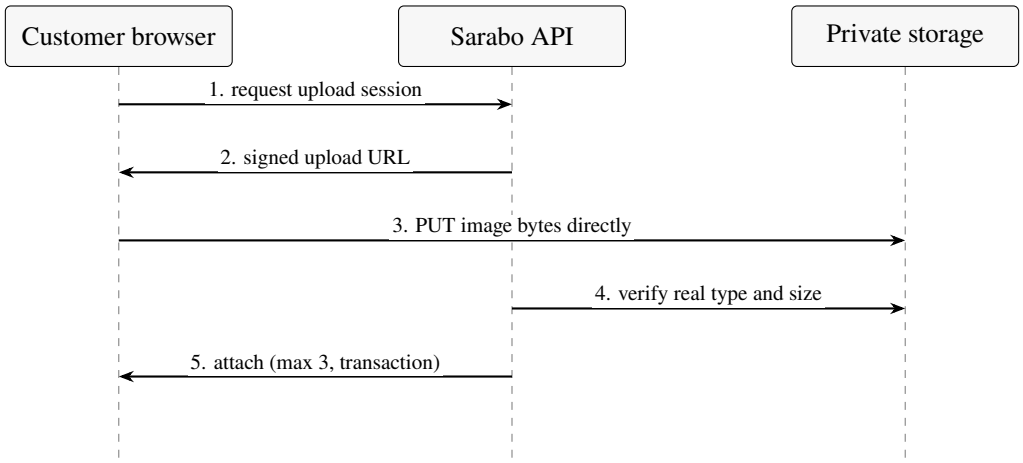


Figure 6.4: Damage-image upload workflow

Chapter 7

AI-Assisted Features

Sarabo uses Google Gemini (model `gemini-3.1-flash-lite`) in four places where it saves users real effort. In every case the AI *suggests* and a person *decides*: nothing the model returns changes a price, a payment or a request state without a human reviewing it first. All four features are switched off by default and need both an explicit setting and an API key; the client asks `GET /ai/status` and hides a feature that is not available.

7.1 AI Customer Support Chat

A chat widget on every page answers visitor and customer questions about how Sarabo works, which services exist and what they usually cost. The prompt is built from the *live* service catalogue (`supportKnowledgeService`), so answers about prices always reflect the current catalogue. Conversation history, message length and answer length are capped, and suggested follow-up buttons (View services, Track a request, Log in, Create an account) are chosen from a fixed list rather than generated as free links.

7.2 AI Request Helper

On the create-request form a customer can describe the problem in their own words (Bangla, English or the common mixed "Banglish") and/or add a photo of the damage. The AI returns a draft: device category, catalogue service, brand, model, a clear English description, a confidence

level, an optional follow-up question and, when a photo was sent, whether it actually shows a device and what is visible. The form fills itself, the customer edits anything, and *Undo* restores what was there before. If the photo shows a device, it is uploaded to the new request automatically after creation, so the customer does not have to pick it twice.

7.3 Technician Price Guide and AI Offer Note

When a technician writes an offer, a price guide (computed without AI) shows the usual range from real offers for the same service in the last 180 days: the 25th percentile of "from" prices to the 75th percentile of "to" prices, and the same for inspection fees. With fewer than three past offers it falls back to the catalogue range. The form gives advisory warnings if an offer is far outside the usual range.

The technician can also ask AI to write the offer note in English or Bangla. The model sees only what the technician already sees (device, brand, model, repair, the customer's problem text and whether photos exist), writes in the technician's voice, describes the likely issue only as a possibility, says the final price comes after inspection, and contains no numbers. Phone numbers (including Bangla digits), emails, links and social handles are stripped from the answer so that contact stays on the platform.

7.4 AI Summary of Admin Reports

The admin Reports page (7, 30 or 90 days) can show a short AI summary: a headline, two to four highlights and one to three suggested actions. The model receives only aggregate figures (requests made, finished and cancelled, success rate, average days, refused demand by region, top devices and repairs, technician workload, inspection-fee money and current attention counts), never names, emails or IDs. The last summary per period is stored, so opening the page costs nothing.

7.5 Safeguards

Table 7.1 summarises how the main risks of LLM features [13] are handled.

All AI tests use fake providers, so the automated suites never call Gemini or spend money.

Table 7.1: AI risks and safeguards

Risk	Safeguard in Sarabo
Invented catalogue items	The request helper must answer with IDs from an enum built from the live catalogue (JSON schema); the server re-checks them and drops unknown or inactive IDs, or a repair belonging to another device.
Invented prices	No AI feature ever sets a price. Estimates come from the catalogue; the offer note contains no numbers.
Invented statistics	Every sentence of the report summary is checked: each number must appear in the facts, be a difference or rounded percentage between periods, a small count up to 10, or a fixed context number. Failing sentences are dropped; if nothing remains, the request fails with 502.
Prompt injection	Customer text is marked as untrusted data in the prompt, and output is forced into a schema.
Personal data leakage	Text containing a card number, password, token or tracking code is refused before any AI call. Models receive no names, emails or addresses. Contact details are stripped from generated notes.
Unbounded cost	Per-user 10-minute and daily limits plus a site-wide daily budget per feature, stored in MongoDB so all serverless instances share them. If the database is unreachable, the feature fails closed.
Bad photos	JPEG/PNG/WebP only, at most 1.5 MB, first bytes must match the type; the client shrinks photos to 1024 px first. The helper's photo is not stored by the AI endpoint.
Provider failure	Timeouts and controlled errors (503 unavailable, 504 timeout, 502 invalid answer); the rest of the form keeps working.

Table 7.2: Default AI usage limits

Feature	Per user / 10 min	Per user / day	Site / day
Support chat (signed in)	20	100	300
Support chat (anonymous)	8	30	(shared)
Request helper	10	30	200
Offer note	10	40	200
Report summary	5	20	50

Chapter 8

Security, Reliability and Data Integrity

8.1 Security Principles

Sarabo is *server-authoritative*. The client decides what to display; the server decides what is allowed. This follows the common REST security guidance that authorization must be enforced at each resource operation [18]. Secrets (database, Stripe, Supabase, Gemini, Gmail, cron) exist only in server environment variables and are never committed or sent to the browser.

8.2 Authentication

Firebase issues ID tokens to signed-in users. Each protected request sends the token, the server verifies it with Firebase Admin, and the caller's email is taken from the verified token only. Endpoints such as `/technicians/me`, `/users/me/role` and the wallet never accept another user's email as input. Customer actions that create requests or payments require a verified email address.

8.3 Role- and State-Based Authorization

Every write endpoint checks both who the caller is and what state the request is in:

- only Admins approve technicians, edit the catalogue, retry refunds, process withdrawals and moderate feedback;

- only an approved, matching Technician can see job details or send an offer, and only the booked Technician can inspect, quote, revise and repair;
- only the owning Customer can choose or reject offers, decide on a quote, pay, cancel, confirm receipt and review;
- a quote can be submitted only after inspection, payment only after approval, revision only from `quote_rejected`, and a paid repair can never re-enter the rejected-quote path.

Requests that do not belong to the caller return the same "not found" answer as missing ones, so their existence is not revealed.

8.4 Financial Integrity

Prices, fees, credits, commissions, balances and withdrawal limits are always derived on the server from stored data. Money is handled in whole taka. Stripe sessions are created from stored amounts; the webhook signature is verified on the raw body [10]; payment processing is idempotent; refunds carry idempotency keys; settlements are frozen once; and withdrawals move from *requested* to *paid* or *rejected* through a single-winner update, so the same withdrawal can never be paid twice.

8.5 Privacy

Public endpoints (tracking, jobs board, service availability) return purpose-built projections without names, emails, addresses, phone numbers, payment data or database IDs. Technicians see a customer's district and problem, but not identity or street address, until booked. Refused-demand counts store a hash instead of the customer's email. Reports about technicians are visible only to admins.

8.6 Private Image Security

The storage bucket is private. Signed upload and read URLs are short-lived and specific to one object. The server checks the real stored type and size rather than trusting the browser, and the

permanent object identifier is never returned by the API.

8.7 Input Validation and Safe Errors

The server validates IDs, state transitions, amounts, text lengths, file types and every mutation's allowed fields; unexpected fields are rejected. User text is stored as plain text and rendered as text. Errors use stable codes (for example `PICKUP_SLOT_FULL`, `NO_TECHNICIAN_AVAILABLE`) instead of raw provider messages, and logs record only the request ID, route, status and a broad category, never request bodies, comments or secrets.

8.8 CORS, Rate Limiting and Abuse Control

The API accepts browser calls only from the configured client origin, which must be `https` in production. Public tracking, the jobs board and every AI feature are rate limited; AI and support limits are stored in MongoDB so they hold across serverless instances. The scheduled job endpoint requires a secret and refuses to run without one.

8.9 Transaction Safety

MongoDB transactions protect the invariants that matter most:

- a request's state and its technician's availability always agree, so a technician cannot be double-booked;
- a payment, the booking it causes and the offers it closes change together;
- a quote transition, its tracking event and its notification apply together or not at all;
- a fee refund is marked pending in the same transaction as the event that causes it.

A late defect (Appendix E) showed a subtle rule: if an insert inside a transaction fails with a duplicate key, MongoDB has already aborted the transaction, so that error must be passed on to let the driver retry the whole transaction, never swallowed.

8.10 Environment Isolation

Development, automated tests and production use different database names, and the code refuses to run tests against the development or production database. Seed and migration scripts default to dry runs and refuse production unless an operator passes explicit one-time flags. Continuous integration runs on a throwaway database and is never given real credentials.

8.11 Legacy Compatibility Boundary

Some stored values come from the original parcel-delivery codebase: the role value `rider` and status values such as `driver_assigned` or `parcel_delivered`. They are treated as frozen contract values behind a presentation layer; everything users see uses repair terminology. Renaming them would require a coordinated data migration and is listed as future work.

Chapter 9

Testing and Evaluation

9.1 Testing Strategy

Testing focused on *domain correctness*: whether the right person can do the right thing at the right time, exactly once. Four layers were used (Figure 9.1).

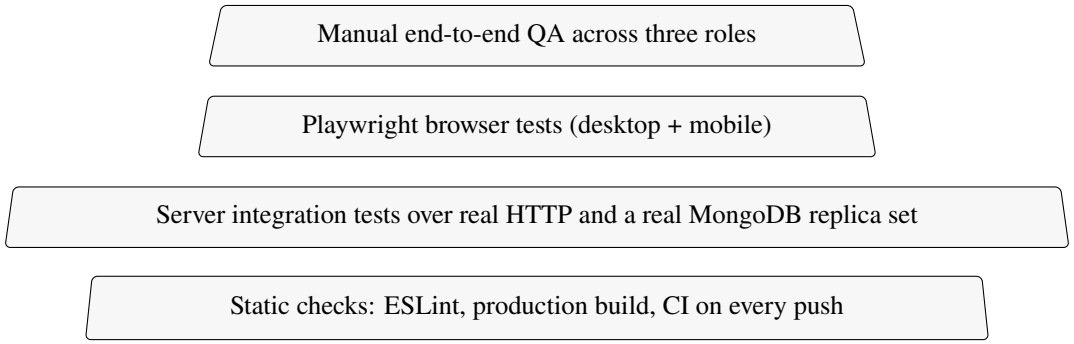


Figure 9.1: Testing layers used in Sarabo

Server integration tests. A custom harness (`test-all.js`) starts its own API server on a random port against an isolated test database, seeds a baseline (role accounts, an approved technician, the catalogue) and exercises endpoints over real HTTP, plus direct service-level checks. Stripe, Supabase, Gemini and email are replaced by injected fakes, so the suite never touches external services or money. Concurrency is tested by firing competing requests at once, for example two admins assigning different technicians to the same request, a cancellation racing a payment, or the same Stripe webhook delivered several times in parallel; each must produce exactly one winner

and a consistent final state.

Browser tests. Playwright runs against a real production build of the client with a faked API, on a desktop and a mobile (Pixel 7) viewport. The tests cover public pages, the jobs board (listing, filtering, sorting, empty and error states), tracking (valid, unknown, invalid codes and server errors), authentication forms and route protection.

Continuous integration. GitHub Actions runs the server suites against a throwaway MongoDB 8 replica set and runs client lint and build on every push and pull request. The latest runs on the main branch of both repositories (11 October 2026, on the final commits) passed.

9.2 Verification Results

For this report every check was run again on 11 October 2026 against the final code (client commit `428ee11`, server commit `0d1e758`). The server suites were run exactly as in CI, using a clean copy of the repository with no local secrets and a temporary in-memory MongoDB replica set. Table 9.1 shows the results.

Table 9.1: Verification results (11 October 2026)

Check	Passed	Failed
Server full suite (`npm test`)	2,595	0
of which re-run on their own: support and AI suite	160	0
of which re-run on their own: data-migration suite	63	0
of which re-run on their own: feedback suite	177	0
Client browser tests (Playwright, 29 scenarios × 2 viewports)	58	0
Client ESLint	no errors	
Client production build	success	
GitHub Actions, latest `main` run, both repositories	success	

The server suite has grown from 1,890 checks at the time of the earlier report (August 2026) to 2,595, mainly through coverage of the job portal, inspection-fee outcomes, pickup scheduling, email, reports and the AI features. The last 15 of these cover the final payout on technician removal.

9.3 Representative Test Scenarios

Table 9.2: Representative test scenarios

ID	Scenario	Expected and verified result
T-01	Request without a token to a protected endpoint	401; no data returned
T-02	Customer calls an admin endpoint	403; nothing changes
T-03	Request in a region with no matching technician	409 `NO_TECHNICIAN_AVAILABLE`; refusal counted once per day
T-04	Customer picks a pickup slot that is already full	409 `PICKUP_SLOT_FULL`; request not saved
T-05	Unapproved or non-matching technician sends an offer	Refused; offer not stored
T-06	Inspection fee above the catalogue cap	Refused by the server
T-07	Fee payment arrives after another technician was booked	Payment recorded and refunded automatically
T-08	Same Stripe event delivered twice, or webhook and success page both fire	Exactly one payment row and one booking
T-09	Customer rejects an offer	Job reposted; technician may send one revised offer only
T-10	Cancel 3 hours vs 1 hour before pickup	Full refund vs fee kept and split 90/10
T-11	Stripe refund call fails	Refund stays pending, shows on admin home, retry succeeds once
T-12	Quote submitted before inspection	Controlled conflict; state unchanged
T-13	Quote rejected, revised and resubmitted	Old quote archived; new round gets its own notification; no transaction hang
T-14	Approved quote with fee already paid	Amount due = quote total minus fee; settlement on the full quote
T-15	Paid request moves on to repair states	Still reported as paid; no second Pay button
T-16	Repair completed but not yet received	Settlement pending; cannot be withdrawn

ID	Scenario	Expected and verified result
T-17	Withdrawal above available balance, or two open withdrawals	Refused; database allows only one open request
T-18	Withdrawal marked paid, then technician asks again	Nothing left to withdraw; balance can never go negative
T-19	Duplicate review, or review before receipt	Refused; one review per repair
T-20	AI draft names an unknown or inactive service	ID dropped by the server
T-21	AI summary sentence with a number not in the facts	Sentence removed before saving
T-22	Problem text containing a card number	Refused before any AI call
T-23	Public tracking and jobs board responses	No names, emails, addresses or payment data
T-24	Seed or migration script pointed at production	Refused without explicit operator override
T-25	Admin removes a technician with 3,000 BDT available	Removal and a 3,000 BDT final bKash withdrawal commit together; with no bKash number on record, removal is refused and nothing changes
T-26	Visitor searches “Ambarkhana” on the Service Areas page	Map zooms to Ambarkhana in Sylhet, not the district centre; “Sylhet” still opens the district

9.4 Manual End-to-End Acceptance

In August 2026 the core lifecycle was accepted by hand in the development environment with three accounts and Stripe test cards: request, assign, accept, inspect, quote, reject, revise, re-quote, approve, pay, repair, complete and confirm receipt, checking at each step that all three roles saw the same state and that the settlement became available only after receipt. For the final submission the

same walkthrough is extended to the marketplace flow (AI-assisted request, offers, offer rejection and repost, inspection-fee payment, approve-and-pay, bKash withdrawal and review) as part of the final improvement phase (Section 5.13).

9.5 Evaluation Against Objectives

Table 9.3: Evaluation against objectives

Obj.	Status	Evidence
O1	Achieved	Firebase identity, server role and state checks (T-01, T-02)
O2	Achieved	Structured V2 form, catalogue, photos, pickup slots, AI helper (T-04, T-20)
O3	Achieved	Region and expertise gate, jobs board, offers and reposting (T-03, T-05, T-09)
O4	Achieved	Fee booking, credit, refunds and split (T-06, T-07, T-10, T-11, T-14)
O5	Achieved	Inspection, quote, rejection and revision (T-12, T-13)
O6	Achieved	Single payment truth, idempotent processing, PDF receipts (T-08, T-15)
O7	Achieved	Progress, evidence, receipt confirmation, reviews and reports (T-16, T-19)
O8	Achieved	41 notification types, email, daily reminders, sanitized tracking (T-23)
O9	Achieved	90/10 settlement, computed wallet, bKash withdrawals, final payout on removal (T-16 – T-18, T-25)
O10	Achieved	Attention list, reports with checked AI summary, catalogue editor, moderation (T-21)
O11	Achieved	2,595 server checks, 58 browser tests, lint, build, CI, live deployment

9.6 Validation Boundary

The automated results above refer to an isolated test environment with fake external providers. Payments run in Stripe test mode, so no real money has moved through the platform, and the system has not yet been used by real customers. Load and performance testing under heavy traffic, and a formal security audit, were outside the scope of this project.

Chapter 10

Limitations and Future Work

10.1 Current Limitations

10.1.1 Manual Technician Payouts

The wallet tracks earnings and withdrawals precisely, but the money itself is sent by an admin using bKash “Send Money”, who then records the transaction ID. The platform does not yet pay technicians automatically through a bKash, Nagad or bank API.

10.1.2 Card Payments in Test Mode

Customer payments use Stripe in test mode. Stripe does not offer local accounts for businesses in Bangladesh, so a real launch would need a local gateway such as SSLCOMMERZ or bKash Payment Gateway behind the same payment service interface.

10.1.3 Pickup Capacity Is Checked, Not Locked

Slot capacity is counted before a request is saved but is not locked, so two customers booking the last place in the same second could both succeed. At the current scale this is acceptable (admins see every pickup time), but a busy service would need an atomic slot counter.

10.1.4 Provisional Catalogue Prices

The 45 services and their estimate ranges are a starting point for demonstration, not prices reviewed or approved by a real repair business. Admins can change them at any time.

10.1.5 Single Active Job per Technician

A technician is either available or busy with one repair. This prevents double booking but does not model technicians who can handle several small jobs per day.

10.1.6 Legacy Internal Values

Some stored role and status values (`rider`, `parcel_delivered`, and others) are inherited from the original parcel-delivery codebase. They are hidden from users, but a dedicated migration would make the data model cleaner.

10.1.7 Not Yet Used by Real Customers

The platform is deployed and tested, but it has not been run with real customers, technicians and money, and it has not undergone load testing or an external security audit.

10.2 Future Enhancements

1. **Local payment gateway and automated payouts**: integrate SSLCOMMERZ or bKash for customer payments and technician disbursement, with reconciliation.
2. **Live technician location**: show the technician's approach on a map on pickup day.
3. **Mobile apps or a Progressive Web App**: offline-friendly technician app with push notifications.
4. **SMS notifications**: many customers in Bangladesh rely on SMS more than email.
5. **Warranty and returns**: post-repair warranty cases with evidence and re-inspection.
6. **Parts inventory**: track parts used, their cost and supplier per repair.

7. **Technician scheduling**: daily capacity, working hours and route-aware assignment.
8. **Smarter matching**: rank offers using ratings, distance, response time and completion history.
9. **Bangla interface**: full Bangla localisation of the user interface, not only of AI input.
10. **Schema clean-up**: retire the remaining legacy values through a planned data migration.

Chapter 11

Conclusion

This project developed Sarabo, an electronics repair service management platform that turns an informal, trust-based process into a tracked, priced and auditable service. Customers can describe a problem in their own language, see an estimate, compare offers from qualified technicians in their area, pay a small inspection fee that counts toward the final price, approve an itemised quote, pay securely and follow every step. Technicians get a steady source of matching jobs, tools that help them price and describe their offers, and a transparent wallet. Admins get a single view of what needs attention and how the service is performing.

The most important engineering outcome is the server-authoritative workflow. An action succeeds only if the caller has the right role, the request is in the right state, and every amount involved is derived from stored data. Combined with MongoDB transactions, idempotent payment processing, idempotent refunds and computed (never stored) balances, this keeps money and state consistent even when requests are retried, delivered twice or raced against each other. The project also shows how generative AI can be added responsibly: constrained output, server-side validation against live data, no AI-set prices, number-checked summaries, privacy filtering and strict usage budgets.

Compared with the version described in the earlier report, Sarabo grew from an admin-assigned repair workflow into a two-sided marketplace with a job portal, inspection fees and refunds, pickup scheduling, reviews, email, reports, four AI features and a production deployment, while its automated server checks grew from 1,890 to 2,595, all passing, alongside 58 browser tests and continuous integration for both repositories.

As a solo final year project, Sarabo provided practical experience in requirements analysis, API and database design, state-machine thinking, role-based security, payment and refund handling, financial accounting rules, integration of external services, applied AI, interface and accessibility design, automated testing, continuous integration and deployment. The limitations identified in Chapter 10 (chiefly a local payment gateway, automated payouts and real-world use) form a clear path for taking Sarabo from a validated academic system to a live service.

References

[1] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, 2000.

[2] Meta Platforms, Inc., “React documentation,” https://react.dev/, accessed: Oct. 2026.

[3] TanStack, “Tanstack query documentation,” https://tanstack.com/query/latest, accessed: Oct. 2026.

[4] Vite Contributors, “Vite documentation,” https://vite.dev/, accessed: Oct. 2026.

[5] Tailwind Labs, “Tailwind CSS documentation,” https://tailwindcss.com/docs, accessed: Oct. 2026.

[6] OpenJS Foundation, “Express documentation,” https://expressjs.com/, accessed: Oct. 2026.

[7] MongoDB, Inc., “MongoDB manual,” https://www.mongodb.com/docs/manual/, accessed: Oct. 2026.

[8] Google, “Firebase authentication documentation,” https://firebase.google.com/docs/auth, accessed: Oct. 2026.

[9] Stripe, Inc., “Stripe payments and checkout documentation,” https://docs.stripe.com/, accessed: Oct. 2026.

[10] ——, “Receive stripe events in your webhook endpoint,” https://docs.stripe.com/webhooks, accessed: Oct. 2026.

[11] Supabase, “Supabase storage documentation,” https://supabase.com/docs/guides/storage, accessed: Oct. 2026.

[12] Google, “Gemini API documentation,” https://ai.google.dev/gemini-api/docs, accessed: Oct. 2026.

[13] OWASP Foundation, “OWASP top 10 for large language model applications,” https://genai.owasp.org/llm-top-10/, accessed: Oct. 2026.

[14] World Wide Web Consortium, “Web content accessibility guidelines (WCAG) 2.2,” W3C Recommendation, https://www.w3.org/TR/WCAG22/, 2023.

[15] Sheba Platform Limited, “Sheba.xyz,” https://www.sheba.xyz/, accessed: Oct. 2026.

[16] Urban Company, “Urban company,” https://www.urbancompany.com/, accessed: Oct. 2026.

[17] Thumbtack, Inc., “Thumbtack,” https://www.thumbtack.com/, accessed: Oct. 2026.

[18] OWASP Foundation, “REST security cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/REST_Security_Cheat_Sheet.html, accessed: Oct. 2026.

Appendices

Appendix A

API Surface

All routes except the public ones require a Firebase ID token. “Role” shows who may call the route (C = Customer, T = Technician, A = Admin, P = public).

Table A.1: Sarabo API endpoints (main families)

Method	Route	Role	Purpose
Public			
GET	`/service-definitions`, `/service-definitions/:id`	P	Active catalogue
GET	`/service-availability`	P	Can a request be served here?
GET	`/pickup-slots`	P	Available pickup windows
GET	`/jobs`	P	Open jobs board
GET	`/public/trackings/:trackingCode`	P	Sanitized tracking
GET	`/ai/status`	P	Which AI features are on
POST	`/stripe-webhook`	Stripe	Signed payment events
GET	`/health`	P	Health check
Repair requests			
GET/POST	`/repair-requests`	C	List own / create
GET	`/repair-requests/:id`	C,T,A	Authorized detail
PATCH	`/repair-requests/:id/pickup-slot`	C	Reschedule pickup
PATCH	`/repair-requests/:id/cancel`	C	Cancel before booking
GET/POST	`/repair-requests/:id/booking-cancellation`, `/cancel-booking`	C	Preview / cancel a booked visit

Method	Route	Role	Purpose
POST	/ai/request-draft	C	AI request helper
Job portal			
GET	/technician/job-portal	T	Matching open jobs with details
POST	/repair-requests/:id/applications	T	Send an offer
GET	/repair-requests/:id/applications/price-guide	T	Price guide
POST	/repair-requests/:id/applications/ai-note	T	AI offer note
POST	/repair-requests/:id/applications/mine/withdraw	T	Withdraw own offer
POST	/repair-requests/:id/applications/:applicationId/accept	C	Choose an offer
POST	/repair-requests/:id/applications/:applicationId/decline	C	Reject an offer (repost)
Inspection, quote, payment, repair			
POST	/repair-requests/:id/inspection	T	Submit inspection
POST	/repair-requests/:id/quote	T	Submit quote
POST	/repair-requests/:id/quote/decision	C	Approve / reject
POST	/repair-requests/:id/quote/revise	T	Revise rejected quote
POST	/repair-requests/:id/quote/cancel-request, /quote/close	T, C	End after a declined quote
GET	/repair-requests/:id/payment-eligibility	C	Can this be paid now?
POST	/repair-requests/:id/checkout-session	C	Start repair payment
GET	/payments, /payments/:transactionId/receipt.pdf	C,A	History and PDF receipt
POST	/repair-requests/:id/repair/start, /progress, /evidence, /complete	T	Repair workflow
POST	/repair-requests/:id/confirm-receipt	C	Confirm hand-over
Technicians, wallet and feedback			
GET	/technicians/me, /technicians/me/reviews	T	Own profile and reviews
PATCH	/technicians/:id/expertise	T,A	Edit expertise
GET	/technician/wallet	T	Wallet balances
POST	/technician/withdrawals	T	Request bKash withdrawal
GET	/repair-requests/:id/feedback	C	Review eligibility and own feedback

Method	Route	Role	Purpose
POST	/repair-requests/:id/review, /technician-reports	C	Review / private report
GET	/notifications, /notifications/unread-count	C,T,A	In-app notifications
Administration			
GET	/admin/repair-requests	A	Paginated, filterable requests
POST	/admin/repair-requests/:id/invite	A	Invite a technician to apply
POST	/admin/repair-requests/:id/missed-pickup/withdraw	A	Technician no-show
POST	/admin/repair-requests/:id/inspection-fee/retry-refund	A	Retry a pending refund
GET	/admin/attention, /admin/reports	A	Attention list and reports
GET/POST	/admin/reports/ai-summary	A	AI summary
GET/PATCH	/admin/service-definitions, /:id	A	Catalogue editor
PATCH	/technicians/:id	A	Approve / reject technician
GET/POST	/admin/withdrawals, /:id/mark-paid, /:id/reject	A	Withdrawals
GET/PATCH	/admin/technician-reviews, /admin/technician-reports	A	Moderation
PATCH	/users/:id/role	A	Change a user's role
GET	/cron/pickup-reminders	Cron	Daily reminders and auto-close

Appendix B

Data Model Summary

Table B.1: Selected fields of a repair request

Field	Meaning
`schemaVersion`	2 for modern requests
`product`	Device category, brand and model snapshot
`service`	Chosen service with its price rule snapshot
`trackingId`	Public tracking code (`SRB-...`)
`deliveryStatus`	Lifecycle state (frozen legacy name)
`pickupSlot`	Chosen two-hour window and region
`damage.images`	Up to three private image identifiers
`repostedAt`, `repostCount`	Job-board reposting after a rejected offer
`technicianId`, `technicianEmail`	Booked technician
`assignmentHistory`	Append-only assignment audit trail
`inspectionPayment`	Inspection fee payment and refund state
`inspectionFeeSettlement`	90/10 split of a kept fee
`inspection`	Findings and detected issues
`quote`, `quoteHistory`	Live quote and archived rejected quotes
`payment`	Repair payment state
`repair`	Progress, completion and evidence
`technicianSettlement`	Frozen commission and receivable
`closedBy`	Who ended a declined request

Appendix C

Financial Rules

$$\text{Repair subtotal} = \text{Labour} + \text{Parts} + \text{Additional charges}$$

$$\text{Amount due} = \max(\text{Quote total} - \text{Inspection fee paid},\ 0)$$

$$\text{Sarabo commission} = \text{round}(10\% \times \text{Repair subtotal})$$

$$\text{Technician receivable} = \text{Repair subtotal} - \text{Sarabo commission}$$

$$\text{Kept inspection fee} : \text{same 90/10 rule, whole taka}$$

All amounts are whole Bangladeshi taka (BDT). The settlement is calculated on the full quote, even when part of it was covered by the inspection fee. A settlement becomes withdrawable only after the customer confirms receipt; a kept inspection-fee share is available immediately.

Table C.1: Worked example

Inspection fee paid at booking	BDT 300
Approved quote (labour 1,200 + parts 1,500 + other 0)	BDT 2,700
Customer pays after approval	BDT 2,400
Sarabo commission (10% of 2,700)	BDT 270
Technician receivable	BDT 2,430

Appendix D

Source and Build Notes

Repositories. Client: `https://github.com/jahidjubaer/sarabo-client`; server: `https://github.com/jahidjubaer/sarabo-server`.

Deployment. Client: `https://sarabo-jahid.web.app`; API: `https://sarabo-server.vercel.app`.

Table D.1: Verified code baseline

Item	State
Client branch / commit	`main` / `428ee11ccb71078c0ee1d693035611afe3914878`
Server branch / commit	`main` / `0d1e758b4763b162d410b916b5c796fb07ee7e2b`
Commits	156 (client), 107 (server), 26 Jul – 11 Oct 2026
Size	about 337 client source files (≈29,000 lines); about 191 server files (≈45,600 lines, including tests)
Server tests	2,595 / 2,595 passed
Browser tests	58 / 58 passed
Lint and build	Passed

Listing D.1: Development commands

```
# client (sarabo-client)
npm install
npm run dev # http://localhost:5173
npm run lint
npm run build
npm run test:e2e # Playwright browser tests
```

```
# server (sarabo-server)
npm install
npm run dev # http://localhost:3000
npm test # full suite, isolated test database
npm run seed:service-definitions -- --confirm-seed
```

Configuration. No secret values appear in this report. Database, Firebase, Stripe, Supabase, Gemini, Gmail and cron credentials are supplied through environment variables, documented by name in each repository's `.env.example`, `README.md` and `SETUP.md`.

Appendix E

Incident Record: Revised Quote Hang

Observed failure. During manual testing in August 2026, submitting a *revised* quote (after the customer had rejected the first one) left the technician's screen on "Working..." indefinitely.

Cause. The "quote submitted" notification used a de-duplication key that was the same for the whole life of a request. The second round therefore hit a duplicate-key error inside the database transaction. The code treated that error as a harmless duplicate and carried on, but MongoDB had already aborted the transaction. The commit then failed, the driver retried the same callback, and the cycle repeated until it timed out, with nothing ever saved.

Fix. The key now includes the quote round, derived on the server:

Listing E.1: Round-aware quote notification key

```
round = quoteHistory.length + 1
key = "repair:" + requestId + ":quote_submitted:" + round
```

In addition, notification creation inside a transaction first checks for an existing key in the same session, and if a race still causes a duplicate-key error, the error is passed on so that the driver restarts the whole transaction cleanly instead of continuing an aborted one. Regression tests cover first and revised submissions, same-round replays and transactional races.

Lesson. Inside a database transaction, an error may already have invalidated the transaction by the time application code sees it. Such errors must be propagated, not swallowed.